

Question 1

Complete

Flag question

The following sets can be encoded as binary strings:

Select one or more:

- ☐ The set of complex numbers, i.e. $\mathbb{C}$.
- ☐ The set of triples of real numbers, i.e., $\mathbb{R} \setminus \text{time} \mathbb{R} \times \mathbb{R}$.
- ☐ The set of pairs of rational numbers, i.e., $\mathbb{Q} \times \mathbb{Q}$.

Question 2

Complete

Flag question

The problem of checking whether two Turing machines are such that the former has less states than the latter is:

Select one or more:

- ☐ In the class **NP**
- ☐ Decidable in exponential time
- ☐ Undecidable.
- ☐ Not in **NP**

Question 3

Complete

Flag question

Which ones of the following inclusions between complexity classes is compatible with our current knowledge (i.e. maybe we do not know if this is true, but it could be)? Here, **NPC** stands for the class of all **NP**-complete problems.

Select one or more:

- ☐ **NP** $\subseteq$ **NPC**.
- ☐ **P** $\supseteq$ **NPC**.
- ☐ **PSPACE** $\subseteq$ **L**.
- ☐ **P** = **EXP**.

Question 4

Complete

[Flag question](#)

A friend of yours claims he designed an algorithm solving the **SAT** and working in polynomial space. You can safely conclude that:

Select one or more:

- ☐ Your friend must be right, because all algorithms solving **SAT** works in polynomial space.
- ☐ Your friend's algorithm might be correct, but one has of course to check the details.
- ☐ Even if your friend is correct, **SAT** would stay **NP**-complete.
- ☐ If your friend is right, then **P** = **NP**.

Question 5

Complete

[Flag question](#)

The representation class of closed intervals in the real line $\mathbb{R}$:

Select one or more:

- ☐ Has infinite VC-dimension.
- ☐ Has a VC-dimension equal to that of axis-aligned rectangles in $\mathbb{R}^2$.
- ☐ Is efficiently PAC-learnable.
- ☐ Has VC-dimension equal to 2

Question 1

Complete

[Flag question](#)

Construct a deterministic TM of the kind you prefer, which decides the following language:

$$L = \{w \in \{0, 1\}^* \mid w \text{ has 1s at all even positions}\}.$$

As an example, the strings 0111, 010, and ε are in the language, while 001 is not in it. Study the time complexity of TM you have defined.

Question 2

Complete

🚩 Flag question

The Fundamental Theorem of Arithmetic states that any natural number greater than 1 can be written uniquely as the product of finitely many prime numbers. You are required to prove that the following problem L is in **NP**. To do that, you can give a TM or define some pseudocode. The language L includes precisely those natural numbers which can be written as the product of exactly 2 prime numbers, each of them occurring one or more times in the product. In doing so, you can assume to have a polytime algorithm deciding primality of natural numbers and working in polynomial time. Examples of numbers in L are $14 = 2 \cdot 7$, but also $72 = 2^3 \cdot 3^2$.

Question 3

Complete

🚩 Flag question

During the course we considered the problem **KNAPSACK**, which takes as input the weights w_i and the values p_i of n objects, and asks whether there exists a subset of the objects such that the total weight is less than a given threshold z , while the total value is greater than a given threshold k . Consider a variation of the problem, called **VARKNAPSACK**, in which the weights are all equal to 1, and study its computational complexity.